

Physical AI Platform

Genesis

次世代 物理シミュレーター

ロボット学習のための汎用物理エンジン

2024年発表 | MIT License | ETH Zurich / CMU

Genesisとは？

ロボティクス・具現化AI研究のために設計された汎用物理シミュレーションプラットフォーム。

特徴	概要
 汎用物理エンジン	剛体・流体・弾性体・布・粒子など、あらゆる物質をシミュレーション
 超高速並列計算	GPU並列処理により従来比 最大4万3000倍 の高速化を実現
 AI/ML対応	強化学習・模倣学習・世界モデル訓練に最適化
 オープンソース	MIT Licenseで公開。研究・商用利用が可能

なぜGenesisが必要か

✗ 従来の課題

- シミュレーターごとに別々のAPIが必要
- 物質ごとに異なるエンジンを使い分け
- シミュレーション速度がボトルネック
- 現実との乖離 (Sim-to-Real Gap)

✓ Genesisの解決策

- 統一APIで全物質・全ロボットを制御
- 単一エンジンで剛体・流体・粒子に対応
- GPU並列で最大 **4.3万倍** の高速化
- pip一発でインストール完了

圧倒的なシミュレーション速度

43,000× — Isaac Gymより高速

RTX 4090 × 26,000 並列環境

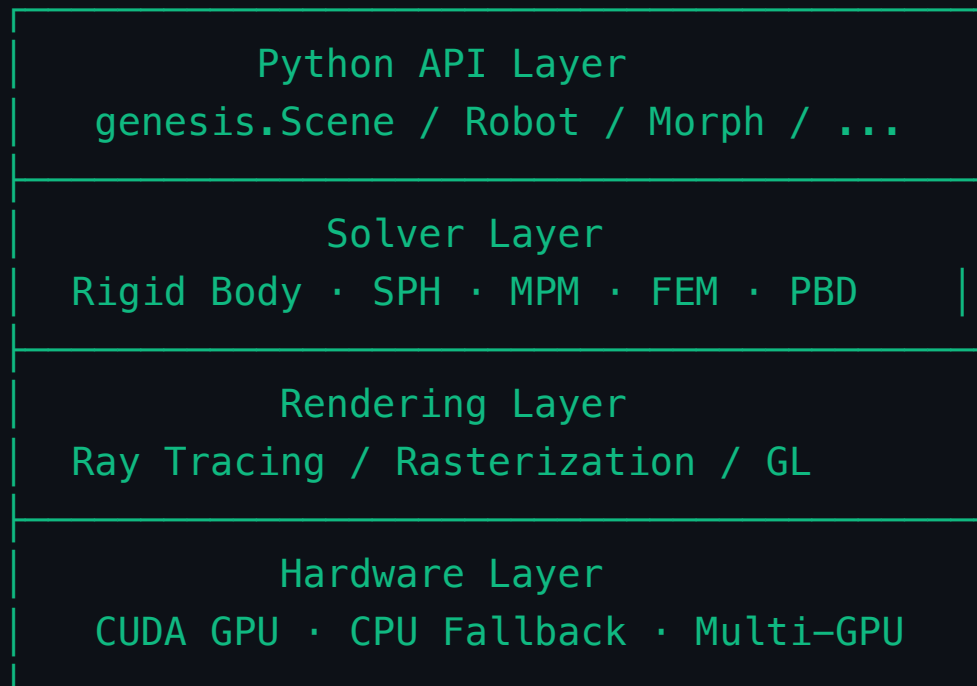
エンジン	相対速度
MuJoCo (CPU)	1×
Isaac Gym (GPU)	~300×
Genesis (GPU)	43,000×

並列環境数: 10,000+ / 初期化: 0.01 sec / スケーラビリティ: 理論上無制限

対応する物理現象

ソルバー	対象	ユースケース
 剛体 (Rigid Body)	衝突・摩擦・関節	ロボット本体のシミュレーション
 流体 (SPH)	液体・ガス	環境との相互作用モデリング
 弾性体 (MPM)	ゴム・粘土	軟体接触のリアル再現
 布 (FEM)	衣服・シート	繊維の物理的振る舞い
 粒子 (PBD)	砂・雪・粉体	バルク材料の取り扱い
 ハイブリッド結合	上記すべて連成	異素材の相互作用を統一処理

アーキテクチャ概要



統一API・統一データ構造で全レイヤーが連携。

ロボット制御での活用

```
import genesis as gs

gs.init(backend=gs.cuda)

scene = gs.Scene(sim_freq=100)

robot = scene.add_entity(
    gs.morphs.MJCF(file="franka.xml")
)

# GPU並列で学習ループ
scene.build(n_envs=1000)
for step in range(10000):
    scene.step()
```

カテゴリ	内容
対応形式	URDF / MJCF / SRDF
センサー	カメラ・深度・力覚・IMU

競合比較

機能	MuJoCo	Isaac Gym	PyBullet	Genesis
物理ソルバー統合	部分的	剛体のみ	剛体のみ	✓ 全種類
GPU並列	✗	✓	✗	✓
流体シミュ	✗	✗	✗	✓
軟体シミュ	✗	✗	✗	✓
速度（相対）	1×	~300×	0.5×	43,000×
ライセンス	Apache 2.0	✗ 非商用	zlib	✓ MIT
インストール	複雑	複雑	pip	✓ pip

クイックスタート

Step 1 — インストール

```
pip install genesis-world
```

Step 2 — シーン作成

```
import genesis as gs
gs.init()
scene = gs.Scene()
```

Step 3 — エンティティ追加

```
robot = scene.add_entity(gs.morphs.MJCF('robot.xml'))
```

Step 4 — 実行

```
gs.render(1000)
```

まとめ

- ⚡ **GPU並列で最大43,000倍の速度** — 従来では不可能な規模の学習が可能
- 🌐 **剛体・流体・弾性体・布・粒子を統一APIで制御** — マルチフィジクス対応
- 🤖 **ロボット強化学習・模倣学習・世界モデル訓練**に最適なプラットフォーム
- 📄 **MIT License・pip一発インストール** — 研究から商用まで自由に利用可能
- 🚀 **Physical AI時代の基盤技術** — Sim-to-Realを加速する次世代エンジン

🔗 github.com/Genesis-Embodied-AI/Genesis

実装事例：SO-101 × Genesis

github.com/oggata/genesis-so101-arm-demo

SO-101 ロボットアームとは

- HuggingFace製OSS 6DOFロボットアーム
- 低コスト（数万円～）で実機購入可能
- URDF形式で物理シミュに直接ロード可
- LeRobot・ACT・Diffusion Policyと連携

このデモで実現すること

1. キューブ把持タスクをPPO強化学習
2. 5～50並列環境で同時学習（GPU加速）
3. OVERHEAD / WRISTの2カメラ視点

システム構成 — GenesisParallelVecEnv

```
# 最近傍キューブまでの距離
dists = np.linalg.norm(cube_poss - eef, axis=1)
nearest_z = cube_poss[nearest_idx, 2]

# 報酬計算（報酬シェイピング）
rewards[i] = (
    -dists[nearest_idx] * 1.5          # ①距離罰則
    + max(0.0, nearest_z - 0.02) * 8.0 # ②高さ報酬
    + (30.0 if success else 0.0)      # ③成功ボーナス
    - 0.01                            # ④時間ペナルティ
)
success = nearest_z > CUBE_LIFT_HEIGHT # 0.07m
```

#	名前	係数	役割
①	距離罰則	× 1.5	EEFとキューブの距離が近いほど高報酬
②	高さ報酬	× 8.0	Z=0.02m以上浮くほど加算
③	成功ボーナス	+30.0	0.07m到達でスパースボーナス付与

報酬シェイピングとは

「AIに何を頑張ってもらいたいかを、段階的なヒントとして報酬に埋め込む設計技術」

✗ スペース報酬だけの場合

- 成功 (+30) 以外は報酬ゼロ → 勾配が流れない
- 何百万ステップも学習が進まない
- 学習曲線：長時間フラット → 突然上昇（不安定）

✓ 報酬シェイピングを使う場合

- 毎ステップ「方向性ある報酬」が得られる
- アームが近づく → 報酬上昇で勾配が流れる
- 学習曲線：序盤から緩やかに上昇 → 安定的に収束

スパース報酬 vs 報酬シェイピング ー 比較

観点	スパース報酬のみ	報酬シェイピングあり
報酬タイミング	成功時のみ（稀）	毎ステップ継続的に取得
学習初期	ほぼ報酬ゼロで停滞	方向性ある勾配が常に存在
探索効率	ランダム探索が必要	目的指向の探索が可能
学習速度	非常に遅い	数倍～数十倍速い
設計難易度	簡単	難しい（副作用に注意）
リスク	学習が進まない	報酬ハッキングのリスク

⚠ 報酬ハッキング：AIが意図しない方法で高スコアを取ることがある。SO-101では「テーブルを揺らして高さを稼ぐ」など。報酬設計は実験と検証の繰り返し。

並列学習の仕組みと学習設定

```
# 1シーンに複数ロボットをグリッド配置
scene.build(
    n_envs=N,
    env_spacing=(0.7, 0.7)
)
```

PPO 学習パラメータ

パラメータ	値
アルゴリズム	PPO (Proximal Policy Optimization)
ポリシー	カスタムMLP (256→256→256)
並列環境数	5～50 (<code>--n_envs</code> で指定)
総ステップ数	300,000 steps
learning_rate	3e-4 (線形スケジュール)
gamma / lambda	0.99 / 0.95 (GAE)